Simulacija dima: paralelizacija grid modela

2. 11. 2026.

Cilj projekta

Gradimo 3D simulaciju dima na mreži ćelija u C++-u, ubrzavamo je korak po korak i mjerimo šta je koja tehnika donijela. Na kraju dim prikazujemo u realnom vremenu u našem OpenGL rendereru.

- **Šta:** 3D grid-based simulacija dima (Eulerov pristup), kocka ćelija npr. 64^3, kasnije 128^3.
- **Kako:** serijska verzija, zatim OpenMP niti i SIMD, a GPU kao moguće proširenje.
- **Šta pokazujemo:** tabele i grafike ubrzanja (speedup) te demo dima u rendereru.

Kako radi simulacija

Prostor s dimom sječemo na male kockice, a svaka ćelija čuva tri stvari: gustoću dima, brzinu zraka i pritisak. Simulacija je petlja: u svakom koraku izračunamo kako dim izgleda 1/60 sekunde kasnije, pa ponovimo.

Jedan korak ima četiri dijela, tim redoslijedom:

1. **Sile.** Glavna je uzgon: topao i rijedak zrak se diže, gust dim teži dolje.
2. **Advekcija brzine.** Strujanje nosi samo sebe.
3. **Projekcija pritiska.** Popravlja brzinu tako da zrak ne nestaje i ne gomila se. Ovo je najskuplji dio.
4. **Advekcija gustoće.** Popravljena struja zraka nosi dim.

Izvor dima dodajemo prije koraka, u nekoliko ćelija pri dnu. Na zidovima kutije brzina okomita na zid je nula, a gustoća i pritisak se kopiraju iz najbliže unutrašnje ćelije. Difuzija (viskoznost) je opcionalna, jer je kod dima mala i često se preskače.

Gustoća, brzina i pritisak

Sve tri veličine računamo samo iz najbližih susjeda ćelije, i to je razlog zašto se simulacija dobro paralelizira.

Veličina	**Šta znači**	**Kako je računamo**
Gustoća ρ	Koliko dima ima u ćeliji	Advekcija: pitamo odakle je zrak došao prije Δt i uzmemo staru gustoću s tog mjesta
Brzina u, v, w	Kuda struji zrak po x, y, z	Sile (uzgon), pa advekcija brzine, pa popravka pritiskom
Pritisak p	Ono što sprečava gomilanje i nestajanje zraka	Iterativno rješavanje (Jacobi), 20 do 80 prolaza po koraku

Gustoća (advekcija). Mjesto odakle je zrak došao rijetko pada tačno u centar ćelije, pa gustoću interpoliramo iz 8 okolnih ćelija (trilinearna interpolacija). Nova vrijednost je uvijek prosjek starih, pa simulacija ne može "eksplodirati" ni za velik Δt. Cijena je blago razmazivanje dima.

$$\rho_{novo}(x) = \rho_{staro}\big(x - \Delta t\, u(x)\big)$$

Pritisak. Zrak se ne može stisnuti: u svaku ćeliju koliko uđe toliko mora i izaći, odnosno divergencija brzine je nula. Poslije sila i advekcije to više ne vrijedi. Gdje se zrak gomila, pritisak raste i gura ga prema susjedima. Svaka Jacobi iteracija uzima prosjek 6 susjeda i oduzima divergenciju:

$$\nabla \cdot u = 0 \qquad\qquad p^{m+1}_{i,j,k} = \frac{p_{i\pm1} + p_{j\pm1} + p_{k\pm1} - \mathrm{div}_{i,j,k}}{6}$$

Na kraju brzinu korigujemo gradijentom pritiska ($u \leftarrow u - \nabla p$). Ispravnost provjeravamo tako što poslije korekcije izračunamo divergenciju i vidimo da je bliska nuli.

Opciono: *vorticity confinement* vraća sitne vrtloge koje numerička difuzija izgladi, pa dim izgleda uskovitlanije.

Paralelizacija: podjela mreže

Mrežu dijelimo na slojeve i svakoj niti dajemo svoj dio, pa svi rade istovremeno. Primjer: 64 sloja i 4 niti, svaka nit dobije 16 slojeva (0–15, 16–31, 32–47, 48–63). To radi jer nova vrijednost ćelije zavisi samo od starih vrijednosti susjeda: svi čitaju iz starog polja, pišu u novo, i niko ne čeka nikoga.

Granice dijelova i trka podataka. Ćelija na kraju jednog dijela treba susjeda koji pripada drugoj niti. Dok samo čitamo staro polje, to nije problem. Problem nastaje kad se vrijednosti mijenjaju na mjestu: nit može pročitati vrijednost koju druga upravo piše, pa rezultat zavisi od slučajnog redoslijeda. To je trka podataka.

Za rješavač pritiska imamo tri opcije:

Metoda	Kako radi	Prednost	Mana
Jacobi	Čita staro polje, piše novo (dva polja)	Potpuno paralelno, bez trke podataka	Sporije konvergira
Red-black Gauss-Seidel	Ažurira na mjestu: prvo crna, pa bijela polja (kao šahovska ploča)	Brža konvergencija, bez trke podataka	Malo složenije
Multigrid / konjugirani gradijent	Rješava na grubim mrežama	Najbrže za velike mreže	Napredna opcija

Počinjemo s Jacobijem, jer je najjednostavniji i potpuno paralelan.

Gdje se troši vrijeme

Očekujemo da faza pritiska troši najviše vremena, jer ima 20 do 80 prolaza kroz cijelu mrežu po koraku. To prvo izmjerimo profilisanjem, pa tek onda paraleliziramo. Razlog je Amdahlov zakon: ubrzanje cijelog programa ograničava dio koji ne ubrzamo.

$$S(P) = \frac{1}{(1-f) + f/P}$$

Ovdje je f dio vremena koji se može paralelizirati, a P broj niti. Primjer: ako pritisak troši 80% vremena i savršeno je paraleliziran ($f = 0{,}8$), na 8 niti dobijamo najviše $S = 1/(0{,}2 + 0{,}1) \approx 3{,}3$. Stvarno f određujemo mjerenjem.

Ubrzanje je ograničeno i propusnošću memorije: svaki Jacobi prolaz čita 7 vrijednosti da bi napisao jednu. Zato uz GFLOP/s mjerimo i GB/s.

Plan po fazama

Radimo iterativno: poslije svake faze provjerimo da dim izgleda ispravno i izmjerimo ubrzanje.

Faza	**Šta radimo**	**Šta dobijamo**
1. Serijska verzija	Jedna nit, ćelija po ćelija; svako polje je poseban float niz; 64^3, poslije 128^3	Referenca za tačnost i osnova za računanje ubrzanja
2. Profilisanje	Mjerimo vrijeme svake faze: sile, advekcija, difuzija, projekcija	Znamo šta se isplati paralelizirati
3. OpenMP niti	Mrežu dijelimo po slojevima, svaka nit svoj dio (`#pragma omp parallel for`)	Mjerenje skalabilnosti na 1, 2, 4… niti
4. SIMD	Vektorizacija unutrašnje petlje po x (AVX2, po potrebi AVX-512), polja poravnata u memoriji	Više ćelija obrađeno po ciklusu procesora
5. Rješavač pritiska	Jacobi, zatim red-black Gauss-Seidel; multigrid kao napredna opcija	Brža konvergencija
6. GPU (proširenje)	Compute shaderi ili CUDA; brzina, gustoća i pritisak ostaju na GPU kao 3D teksture	Bez stalnog prebacivanja podataka CPU $\leftrightarrow$ GPU
7. Demo u rendereru	Gustoću svaki frame šaljemo u renderer kao 3D teksturu; dim crtamo ray marchingom kroz kocku	Dim u realnom vremenu

Renderer je naš opengl-forward-renderer (C++20, OpenGL 4.6 Core, CMake). Prema README-u već ima HDR, tone mapping i bloom, ali ne spominje prozirnost, pa volumetrijski prolaz pišemo sami: dim crtamo poslije neprozirnih objekata, a prije tone mappinga. OpenGL 4.6 podržava 3D teksture, pa nam ne treba ništa dodatno.

Podjela posla i mjerenje

Posao dijelimo na troje, a svi koriste isti interfejs `SmokeSolver::step(dt)`:

1. Serijski solver i tačnost.
2. Mjerenje, SIMD i OpenMP.
3. Rješavač pritiska i veza s rendererom.

Kako mjerimo. Ubrzanje = vrijeme serijske verzije / vrijeme paralelne verzije, za 1, 2, 4… niti. U izvještaju objašnjavamo zašto ubrzanje nije idealno (memorija, granice dijelova, Amdahlov zakon).

Tačnost provjeravamo tako što poslije korekcije pritiska $\max |\nabla \cdot u|$ mora biti blizu nule, a razlika paralelne i serijske verzije mora biti u okviru tolerancije u pokretnom zarezu (kao `verify()` iz Lab1).

Literatura

- J. Stam, Stable Fluids, SIGGRAPH 1999: semi-Lagrangian advekcija i projekcija.
- J. Stam, Real-Time Fluid Dynamics for Games, GDC 2003: kratka C implementacija 2D solvera.
- R. Fedkiw, J. Stam, H. W. Jensen, Visual Simulation of Smoke, SIGGRAPH 2001: uzgon, temperatura i vorticity confinement.
- R. Bridson, Fluid Simulation for Computer Graphics, kursne bilješke.
- M. Harris, Fast Fluid Dynamics Simulation on the GPU, GPU Gems, pogl. 38: Jacobi i mapiranje na paralelni hardver.
- K. Crane, I. Llamas, S. Tariq, Real-Time Simulation and Rendering of 3D Fluids, GPU Gems 3, pogl. 30: 3D dim i rendering volumena.
- A. McAdams, E. Sifakis, J. Teran, A Parallel Multigrid Poisson Solver for Fluids Simulation on Large Grids, 2010.
- Smoke Simulation for Fire Engineering using a Multigrid Method on Graphics Hardware.
- G. M. Amdahl, Validity of the single processor approach to achieving large scale computing capabilities, AFIPS Spring Joint Computer Conference, 1967.